Manual del Programador - TALLER PRÁCTICO DE REPASO Y CONSOLIDACIÓN EN C++

Luca Di Guglielmo Villa
E.E.S.T N°1 de Vicente López
Curso: Laboratorio de Programación 5°3°
Profesor: Ganduglia Yamil & York Elias Mansilla Muñoz
Fecha: 15/10/2026

Introducción

En este manual se explicará la lógica atras de los 3 retos de código en el trabajo ‘Taller práctico de repaso y consolidación en C + +

Desarrollo

Reto 1:

```
// --- RETO 1: RECURSIVIDAD ---
cout << "\n=== RETO 1: SUMA RECURSIVA ===" << endl;
int num = 5;
cout << "Suma acumulada de 1 hasta " << num << ": " << sumaRecursiva(num) << endl;
```

```
// Desarrollo de la función recursiva con caso base garantizado
int sumaRecursiva(int n) {
    if (n <= 0) return 0; // Caso base obligatorio para prevenir Stack Overflow
    return n + sumaRecursiva(n - 1); // Llamada recursiva
}
```

Cada llamada autorreferencial reserva un frame en la pila de memoria (Stack). El Caso Base (if (n <= 0) return 0;) detiene la cadena de llamadas. Si se omitiera el caso base, la función generaría un desbordamiento de pila (Stack Overflow) al agotar el espacio reservado en la RAM.

Reto 2:

```
    // --- RETO 2: ARRAYS Y BÚSQUEDA SECUENCIAL ---
cout << "\n=== RETO 2: BÚSQUEDA EN MEMORIA CONTIGUA ===" << endl;
    const int TAM = 10;
    int vectorDatos[TAM] = {12, 45, 78, 90, 122, 5, 18, 33, 67, 99};
    int numeroBuscar = 18;
    bool hallado = false;

    for (int i = 0; i < TAM; i++) {
        if (vectorDatos[i] == numeroBuscar) {
            cout << "[SISTEMA] Valor " << numeroBuscar << " hallado en el indice [" << i << "]" << endl;
            cout << "          Direccion RAM Hexadecimal: " << &vectorDatos[i] << endl;
            hallado = true;
            break; // Interrupción optimizada del ciclo O(n)
        }
    }
```

El arreglo vectorDatos ocupa celdas contiguas en la memoria RAM, donde cada entero (int) está separado por 4 bytes. Al aplicar la instrucción break, se interrumpe inmediatamente el ciclo al hallar el valor deseado (en este caso, 18 en el índice [6]), optimizando la complejidad temporal O(n) al evitar reiteraciones innecesarias del microprocesador.

Reto 3:

```cpp
// --- RETO 3: INTERCAMBIO CON PUNTEROS ---
cout << "\n=== RETO 3: INTERCAMBIO DE CELDAS ===" << endl;
int x = 100, y = 500;
cout << "Valores previos  -> X: " << x << " | Y: " << y << endl;

// Invocación pasando direcciones de memoria
intercambiarValores(&x, &y);

cout << "Valores actuales -> X: " << x << " | Y: " << y << endl;
cout << "=================================================" << endl;

return 0; // Código 0: Ejecución exitosa
```

```cpp
// Desarrollo del intercambio físico mediante desreferenciación
void intercambiarValores(int* ptrA, int* ptrB) {
    int temporal = *ptrA;
    *ptrA = *ptrB;
    *ptrB = temporal;
}
```

Se utiliza pasaje por dirección (&x, &y) y desreferenciación con punteros (int* ptrA, *ptrA). La variable temporal es fundamental para resguardar temporalmente el dato de la celda de memoria apuntada por ptrA, garantizando que no se pierda al copiar el valor de ptrB sobre ella.

Conclusión

En estos retos se demuestran las capacidades de C + + y enseña conceptos importantes, entre ellos, la búsqueda de datos en vectores, manipulación de datos usando punteros y llamadas autorreferenciales

Referencias:

Luca Di Guglielmo

Github: https://github.com/specIux/actividad-6-LPR

Gitlab: https://gitlab.com/specIux/actividad-6-lpr

Laboratorio de Programación 5°3°

Actividad: 📘📺🛠️🖋️ ACTIVIDAD 6: TALLER PRÁCTICO DE REPASO Y CONSOLIDACIÓ…